

匠芯智修 ——面向汽车总装与数控机床设备的  
智能检修系统  
软件功能设计

2026 年 8 月

## 目录

|                           |    |
|---------------------------|----|
| 第一章 引言 .....              | 1  |
| 1.1 编写目的 .....            | 1  |
| 1.2 项目背景 .....            | 1  |
| 1.3 设计原则 .....            | 2  |
| 第二章 系统总体架构设计 .....        | 4  |
| 2.1 总体架构分层 .....          | 4  |
| 2.2 前端交互层 .....           | 5  |
| 2.3 API 服务层 .....         | 6  |
| 2.4 业务逻辑层 .....           | 6  |
| 2.5 AI 能力层 .....          | 7  |
| 2.6 数据持久层 .....           | 8  |
| 第三章 核心功能模块详细设计 .....      | 9  |
| 3.1 多模态智能问答模块设计 .....     | 9  |
| 3.1.1 模块功能概述 .....        | 9  |
| 3.1.2 核心处理流程 .....        | 9  |
| 3.1.3 会话上下文管理 .....       | 10 |
| 3.1.4 流式输出实现 .....        | 11 |
| 3.2 知识图谱模块设计 .....        | 11 |
| 3.2.1 模块功能概述 .....        | 11 |
| 3.2.2 图谱构建流程 .....        | 12 |
| 3.2.3 可视化交互设计 .....       | 12 |
| 3.3 动态 SOP 作业指引模块设计 ..... | 13 |
| 3.3.1 模块功能概述 .....        | 13 |
| 3.3.2 步骤动态生成逻辑 .....      | 13 |
| 3.3.3 状态流转与闭环设计 .....     | 14 |
| 3.4 检修工单归档与管理模块设计 .....   | 14 |
| 3.4.1 模块功能概述 .....        | 14 |
| 3.4.2 工单自动生成逻辑 .....      | 15 |
| 3.4.3 分级提报与审计 .....       | 15 |
| 3.5 手册与知识库管理模块设计 .....    | 15 |
| 3.5.1 模块功能概述 .....        | 15 |
| 3.5.2 手册解析与切块 .....       | 16 |
| 3.5.3 同步与索引构建流程 .....     | 16 |
| 3.5.4 申请审核流程 .....        | 17 |
| 3.6 用户与权限管理模块设计 .....     | 17 |
| 3.6.1 模块功能概述 .....        | 17 |
| 3.6.2 权限体系设计 .....        | 17 |
| 3.6.3 账号管理流程 .....        | 18 |
| 3.7 移动端支持模块设计 .....       | 19 |
| 3.7.1 模块功能概述 .....        | 19 |
| 3.7.2 功能范围与接口复用 .....     | 19 |
| 3.7.3 数据同步与权限适配 .....     | 20 |

|                         |    |
|-------------------------|----|
| 3.8 国产化适配与部署模块设计 .....  | 20 |
| 3.8.1 模块功能概述 .....      | 20 |
| 3.8.2 架构兼容性设计 .....     | 21 |
| 3.8.3 信创环境测试与验证 .....   | 21 |
| 第四章 接口设计 .....          | 22 |
| 4.1 前后端 API 接口设计 .....  | 22 |
| 4.1.1 智能对话类接口 .....     | 22 |
| 4.1.2 知识库类接口 .....      | 23 |
| 4.1.3 用户管理类接口 .....     | 25 |
| 4.1.4 系统监控类接口 .....     | 26 |
| 4.2 模块间内部接口设计 .....     | 27 |
| 第五章 非功能性设计 .....        | 29 |
| 5.1 性能设计 .....          | 29 |
| 5.1.1 异步任务处理机制 .....    | 29 |
| 5.1.2 检索性能优化 .....      | 29 |
| 5.1.3 前端渲染优化 .....      | 29 |
| 5.2 可靠性设计 .....         | 30 |
| 5.2.1 多级容错降级机制 .....    | 30 |
| 5.2.2 数据备份与恢复策略 .....   | 30 |
| 5.2.3 日志与监控 .....       | 30 |
| 5.3 可扩展性设计 .....        | 31 |
| 5.3.1 插件化模型接入 .....     | 31 |
| 5.3.2 可扩展的节点与文档类型 ..... | 31 |
| 5.3.3 部署架构可扩展 .....     | 31 |
| 5.4 安全设计 .....          | 32 |
| 5.4.1 身份认证与授权 .....     | 32 |
| 5.4.2 数据安全 .....        | 32 |
| 5.5 国产化环境适配设计 .....     | 32 |
| 5.5.1 架构兼容性约束 .....     | 32 |
| 5.5.2 操作系统内核依赖 .....    | 33 |
| 5.5.3 网络与通信约束 .....     | 33 |

# 第一章 引言

## 1.1 编写目的

本文档用于说明“匠芯智修”当前版本的软件功能设计与实现方案，重点覆盖系统总体架构、核心业务模块、接口组织、数据存储及非功能性设计。文档以现有代码实现为基础，对各层职责、关键处理流程和模块间协作关系进行统一描述，为开发、测试、部署和评审提供一致的技术依据。

通过本设计文档，可明确前端工作台、FastAPI 服务、RAG 与 Agent 诊断、知识库与知识图谱、SOP、维修记录、权限管理及系统监控之间的边界与调用关系。文档同时记录当前版本的关键约束与部署方式，便于后续定位问题、维护接口并开展增量迭代。

## 1.2 项目背景

数控机床及其控制系统具有设备型号多、故障信息分散、报警代码专业性强等特点。现场检修往往需要同时查阅设备手册、报警说明和历史维修经验，并结合故障图片、设备型号与现场现象进行综合判断。传统依赖人工翻阅资料和个人经验的方式存在检索链路长、知识复用不足、诊断过程难以标准化等问题，也不利于企业形成持续积累的维修知识资产。

针对上述需求，本项目构建面向 CNC 设备的智能故障诊断与知识服务平台。系统将多模态输入、案例优先检索、RAG 检索增强生成、

LangGraph Agent 多工具诊断、知识图谱、动态 SOP 与维修记录管理整合到统一工作流中，并通过知识同步机制将手册、审核案例和人工修正持续纳入检索体系，实现“故障输入—知识检索—智能诊断—作业执行—维修归档—知识回流”的业务闭环。

### 1.3 设计原则

系统遵循“知识约束、智能协同”的设计原则。标准故障由 RAG 路径快速完成知识召回与回答生成，复杂故障由 LangGraph Agent 根据任务状态调用案例库、维修知识、知识图谱和 SOP 工具进行多步分析。诊断结论尽量建立在企业知识与历史案例基础上，并保留来源信息，降低通用模型脱离业务知识进行推断的风险。

架构设计遵循“分层解耦、模块独立”的原则。前端、API、业务服务、AI 能力和数据存储通过清晰接口协作；模型、数据库、向量库及知识目录均由配置统一管理。对话、知识同步、SOP、维修记录、用户与权限等能力分别封装，减少跨模块直接依赖，便于单独测试与维护。

业务设计强调“作业闭环、过程可追溯”。系统不仅输出故障分析，还将诊断结果转换为与会话绑定的 SOP；首版有效 SOP 生成后保持步骤结构稳定，后续主要更新执行状态与备注。维修结束后形成维修记录和报告，符合条件的实际案例可回流动态知识库，持续提升后续检索与诊断的可复用性。

安全与运维设计遵循“身份可验、权限可控、状态可查”的原则。用户登录后由后端签发 JWT，管理接口在 JWT 鉴权基础上兼容旧版 X-Admin-Token；管理端可查看数据库、知识库、向量索引和模型服务状态。对于模型或向量服务异常，系统保留非流式生成、关键词检索及本地索引等降级路径，避免单一外部服务异常直接中断基础检索能力。

## 第二章 系统总体架构设计

### 2.1 总体架构分层

系统保持前后端分离的五层设计，自上而下分为前端交互层、API 服务层、业务逻辑层、AI 能力层和数据持久层。前端负责检修工作台与管理界面，FastAPI 负责请求接入和鉴权，业务层组织 RAG、Agent、知识同步、SOP 与维修记录流程，AI 层提供语言模型、视觉理解和向量嵌入能力，数据层负责 MySQL、知识文件和向量索引等持久化资源。

用户请求由前端进入 API 层后，根据场景进入 RAG 直出或 Agent 诊断路径。业务服务可继续调用视觉分析、案例检索、维修手册检索、图谱查询及 SOP 状态管理，并将结果通过 SSE 或 JSON 返回前端；会话、SOP、维修记录等状态同步写入持久层。各层通过标准接口协作，使诊断能力和业务状态能够独立演进。

当前部署由 Vue/Vite 前端、FastAPI 后端、MySQL 数据库、知识文件目录及向量检索服务共同组成。DashVector 用于云端向量检索，同时保留 FAISS 本地索引；模型、数据库及向量服务地址均通过环境变量配置。项目已形成 LoongArch64 环境的运行方案，部署时优先采用目标系统的原生 Node.js、Python 与数据库组件，避免对固定 x86 运行镜像形成依赖。



图 1 系统架构图

2.2 前端交互层

前端交互层基于 Vue 3、TypeScript、Pinia 与 Element Plus 构建，负责用户登录、检修对话、会话切换、SOP 展示、维修报告以及管理端操作。界面以工业检修任务为主线组织信息，既支持故障文本和图片输入，也能够实时呈现模型输出、工具调用、知识来源和作业状态。

检修工作台采用三栏结构：左侧管理检修会话，中部承载多模态消息流与输入区，右侧集中展示 SOP 和维修报告。管理端按功能划分用户与权限、手册文件、维修案例、检修记录、维修总结、知识图谱和系统监控等面板，使一线作业与后台知识运营在同一 Web 系统中完成。

Pinia 统一维护会话、当前任务、冻结 SOP、提报状态及用户权限等全局状态；流式对话通过 SSE 持续更新消息与工具调用信息。



知识图谱组件当前通过 `iframe` 嵌入后端 `/kg` 页面，由后端页面调用知识图谱接口并使用 `ECharts` 完成力导向图渲染。

## 2.3 API 服务层

API 服务层以 `FastAPI` 为核心，统一挂载智能对话、Agent、知识库、维修记录、用户权限和系统监控等路由。接口层负责请求参数校验、文件接收、身份鉴权和错误转换，将具体检索、诊断及持久化逻辑交由对应服务模块处理，并通过自动接口文档降低联调成本。

普通数据接口以 `JSON` 作为主要交换格式，图片上传采用 `multipart/form-data`，流式诊断采用 `SSE`。`/chat/stream` 提供 `RAG` 流式问答，`/chat/agent` 提供 `Agent` 多工具诊断；知识、维修、用户及监控接口按业务域拆分，避免单一接口承担过多职责。

鉴权由 `FastAPI` 依赖统一处理。用户登录后获得 `JWT`，可通过 `Authorization` 请求头或 `Cookie` 携带；管理员校验优先验证 `JWT`，同时保留 `X-Admin-Token` 作为兼容路径。跨域配置限定本地前后端开发地址，生产部署时可根据实际域名调整白名单。

## 2.4 业务逻辑层

业务逻辑层负责组织系统核心规则，主要包括 `RAGChain`、`LangGraph Agent`、`KnowledgeSyncService`、`SopService`、维修记录服务、会话与记忆服务以及用户和权限组服务。`API` 层仅负责接入与返回，

具体的检索策略、状态流转、知识同步和持久化操作均在业务服务内部完成。

诊断流程采用双路径协同：**RAGChain** 对标准故障执行案例优先检索、视觉增强、关键词/向量/报警代码召回、结果融合与知识图谱扩展；复杂问题可进入 **Agent**，由智能体按需调用案例库、维修知识库、**MySQL** 图谱和 **SOP** 管理工具。知识同步负责将手册、审核案例和人工修正统一写入知识文件、图谱及向量索引。

**SOP**、维修记录、用户权限等业务状态均通过独立服务维护。**SOP** 服务负责首版冻结与步骤状态更新，维修记录服务负责报告、记录和知识回流，用户服务负责账号、权限组和额外权限计算。模块间仅通过明确的数据结构和服务方法交互，减少接口层与数据层的直接耦合。

## 2.5 AI 能力层

AI 能力层包含大语言模型、视觉理解、向量嵌入以及 **Agent** 工具编排能力。上层业务通过统一配置调用不同模型服务，避免在具体业务代码中固定模型地址和密钥；视觉结果被转换为文本描述与检索线索，向量嵌入用于语义召回，语言模型负责诊断生成与 **Agent** 决策。

当前默认配置采用 **OpenAI** 兼容接口，聊天模型、视觉模型与 **Embedding** 模型分别由 **LLM\_MODEL**、**VISION\_MODEL** 和

EMBEDDING\_MODEL 管理；默认配置中包含 qwen3.7-plus、qwen-vl-plus 与 text-embedding-v3。管理端支持读取和更新聊天、视觉、Embedding 以及语音相关模型配置，使模型服务能够在不修改业务代码的情况下调整。

AI 调用设置了必要的降级路径。RAG 流式生成异常时可回退至非流式生成；当模型结果不可用时，可直接返回已检索到的知识片段作为参考。向量检索异常时，关键词检索仍可继续工作，知识同步同时维护 DashVector 与 FAISS 索引，为不同部署条件提供备用检索路径。

## 2.6 数据持久层

数据持久层采用关系型数据库、文件知识库和向量索引组合存储，不同数据按结构和访问方式分别管理。该设计既保留业务数据的事务性，也便于维修手册、动态知识及向量索引独立更新。

MySQL 主要保存用户、会话、消息、SOP 版本、维修记录及结构化知识图谱等数据；维修手册切块、审核案例、人工修正和同步状态以知识目录中的 JSON 文件组织。语义检索优先使用 DashVector，同时维护 FAISS 本地索引。知识图谱接口以 MySQL 图谱为主要来源，并保留 JSON 回退接口。

上层业务通过服务层访问数据，避免直接依赖具体存储实现。例

如图谱查询在 MySQL 不可用时可回退至 JSON 数据，模型与向量服务也由配置统一切换。由此可在不改变核心业务流程的前提下调整数据库、索引或模型服务配置。

## 第三章 核心功能模块详细设计

### 3.1 多模态智能问答模块设计

#### 3.1.1 模块功能概述

多模态智能问答模块是检修工作台的核心入口，支持故障文本、设备型号与现场图片输入。系统同时提供 RAG 直出和 LangGraph Agent 两类诊断方式：标准问题优先通过企业知识快速回答，复杂问题则由 Agent 结合历史案例、维修手册、知识图谱和 SOP 状态进行多工具分析。

#### 3.1.2 核心处理流程

请求进入后首先整理用户问题、设备型号和图片信息。纯文本流式请求进入 RAG 检索链；多模态上传接口会将图片转换为可供视觉模型处理的数据，并在当前实现中转入 Agent 路径。视觉服务提取的故障描述与关键词会补充到检索问题中，设备型号则用于缩小知识匹配范围。

RAG 检索首先查询维修案例库，若命中高相关历史案例，则优先

复用既有处理经验，并检索相关手册内容进行规范补充；未命中时再执行向量检索、关键词检索和报警代码提取，并在结果融合后利用知识图谱扩展关联文档。该流程兼顾历史经验、语义相关性与精确报警信息。

多路召回结果按文档标识去重后统一评分。当前 RAGChain 对归一化关键词得分、向量相似度和图谱增强项分别采用 45%、45% 和 10% 的权重，按综合得分选取 TopK 结果作为生成上下文；案例优先路径命中时则直接以案例上下文为主，并由维修手册提供补充验证。

生成阶段根据路径调用语言模型输出故障现象、原因分析、解决方案和经验总结等内容。RAG 路径同步返回引用来源与匹配得分；Agent 路径则在流式文本之外返回工具调用事件，并可即时推送 SOP 状态，使用户能够看到诊断所调用的知识来源和作业流程变化。

### 3.1.3 会话上下文管理

会话按 session\_id 隔离并保存历史消息，便于任务回溯。RAG 路径主要依据当前问题和最新检索结果生成回答；Agent 路径会注入最近的会话消息，并在已有 SOP 时额外注入冻结的步骤上下文，使后续诊断能够结合当前检修进度。系统还可根据对话内容更新用户长期记忆，用于后续任务中的辅助检索。

### 3.1.4 流式输出实现

流式输出统一采用 SSE。根据诊断路径不同，数据流可包含文本、来源、工具调用、SOP 状态、错误信息与结束标识等事件，前端依据事件类型增量更新界面。

`type: 'text'`——携带增量诊断文本，前端持续追加到当前助手消息。

`type: 'sources'、'tool_start'、'tool_end'`——分别返回知识来源及 Agent 工具调用过程。

`type: 'sop_version'、'sop_state'、'error'、'done'`——用于同步 SOP、异常状态与流结束标识。

前端通过 `sendChatMessageStream` 等异步方法持续消费 SSE 数据，并将文本、来源、工具调用与 SOP 状态写入当前会话。流式协议将长耗时的诊断过程拆分为可连续展示的事件，减少用户等待完整结果的空白时间。

## 3.2 知识图谱模块设计

### 3.2.1 模块功能概述

知识图谱模块负责组织设备、部件、故障、原因、解决方案及维修文档之间的关联关系，并为检索和 Agent 诊断提供结构化知识。

当前系统同时保留 JSON 图谱和 MySQL 结构化图谱：前者用于知识

同步与回退，后者用于更细粒度的故障—原因—方案查询。

### 3.2.2 图谱构建流程

知识同步时，系统首先根据手册切块构建设备、文档、部件、故障和标签等基础节点及关系，生成 JSON 图谱。对于需要结构化推理的知识，可通过 TripleExtractor 对文档进行批量三元组抽取，并经过校验、去重后写入 MySQL 图谱表，形成更明确的故障、原因和解决方案关系。

图谱数据写入后由 GraphDBService 提供全量图谱、节点详情和统计查询，Agent 可调用 MySQL 图谱查询工具补充故障原因与处理链路。若 MySQL 图谱不可用，/knowledge/graph 可回退到 JSON 图谱，保证基础可视化仍可使用；系统另提供 Neo4j AuraDB 查询接口作为扩展图谱路径。

因此，知识图谱在系统中不仅用于展示，还参与故障关联扩展与 Agent 诊断。基础 JSON 图谱随知识库同步更新，结构化图谱可通过管理员触发的三元组抽取接口更新，两类图谱共同承担知识组织、关联查询和异常回退职责。

### 3.2.3 可视化交互设计

前端知识图谱组件通过 iframe 嵌入后端 /kg 页面，页面使用 ECharts graph 系列进行力导向布局。节点根据类型和连接度设置显示

样式，支持缩放、拖拽、悬停提示和邻接高亮；当节点数量超过 800 时，页面按连接度保留高关联节点，控制一次渲染规模。

图谱支持按设备、部件、故障、原因和解决方案等实体类型筛选，并同步展示节点数、关系数和类型数。筛选时后端根据 `entity_type` 返回目标节点及关联关系，帮助用户从完整知识网络快速聚焦到某类故障实体或处理链路。

### 3.3 动态 SOP 作业指引模块设计

#### 3.3.1 模块功能概述

动态 SOP 模块将诊断结果转换为与检修会话绑定的标准化步骤，并将步骤结构和执行状态持久化到 MySQL。SOP 不再仅依赖前端临时解析，而是由后端服务统一管理版本、步骤类型、安全提示和执行状态，使作业指引能够跨页面刷新和会话恢复。

#### 3.3.2 步骤动态生成逻辑

首次生成有效 SOP 时，系统从诊断文本中识别操作步骤，并将操作规范、安全要求和警告信息与普通步骤分离保存。SopService 在 `save_version` 中设置硬守卫：一旦当前会话已有非空步骤，后续生成结果不再覆盖原有 SOP 结构，从机制上保证同一检修任务的作业步骤保持稳定。



后续对话会把上一轮 SOP 状态注入 Agent，上层提示明确要求保持步骤标题和描述不变。系统能够识别“前几步已完成”“第几步进行中”等自然语言反馈，并批量更新对应步骤状态与备注；切换会话时则按 `session_id` 加载各自的 SOP，避免多任务之间状态串联。

### 3.3.3 状态流转与闭环设计

每一步保存 `step_status` 与 `step_note`，可按单步或批量方式更新为待执行、进行中或已完成状态。前端根据最新 SOP 状态刷新步骤展示，管理与诊断逻辑均以数据库中的最新版本为准，保证状态更新具备一致的数据来源。

当步骤执行完成后，当前任务可继续进入维修报告和维修记录环节。SOP 与会话、维修报告共同构成诊断过程的关键追踪信息，使一次故障从模型建议延伸到实际执行结果，并为后续记录归档和知识回流提供依据。

## 3.4 检修工单归档与管理模块设计

### 3.4.1 模块功能概述

检修工单模块现已扩展为维修记录与报告管理，用于保存一次故障处理的关键业务数据。系统可记录设备型号、故障类型、故障描述、故障原因、维修方案、更换配件、维修人员、维修时间、处理状态及是否解决等信息，并按用户或全局范围进行查询。

### 3.4.2 工单自动生成逻辑

维修报告通过 `report_order_id` 与对应维修记录关联，完整报告数据持久化到 MySQL。维修记录支持新增、修改、删除和分页查询，并提供 CSV 导出；记录创建或更新后还可触发后台知识同步任务，使检修结果不再停留于一次性工单，而能进入后续知识沉淀流程。

### 3.4.3 分级提报与审计

具备 `submit_report` 权限的用户可提交维修报告，系统保存提报状态并防止同一工单重复提交。管理端可统一查看已提交报告和全局维修记录，结合设备、故障、维修人员及时间等字段开展复盘和审计；相关接口继续受用户权限和管理员鉴权约束。

有效维修记录可被标记为已同步，并自动转换为包含设备型号、故障类型、原因、描述和维修方案的维修案例，写入动态知识库后自动审核通过。下一次知识库同步时，该案例会进入统一知识文件和向量索引，形成“诊断—执行—记录—案例—再检索”的知识闭环。

## 3.5 手册与知识库管理模块设计

### 3.5.1 模块功能概述

手册与知识库管理模块负责维修资料、动态案例和人工修正的统一组织。管理员可维护设备手册并触发同步，普通用户可提交手册申

请；同步服务将正式手册、审核通过的案例和修正条目合并为统一知识文档，并同步更新图谱和向量索引。

### 3.5.2 手册解析与切块

系统支持 PDF、DOCX、XLS 与 XLSX。PDF 优先通过 pypdf 提取文本，当检测到文字层不足时启用 pdf2image 与 Tesseract OCR 回退；长文本按 `CHUNK_SIZE=900`、`CHUNK_OVERLAP=120` 进行滑动切块。DOCX 按标题层级组织内容，XLS/XLSX 则按报警代码表逐行转换为标准化知识条目。

解析结果统一补充文档 ID、来源文件、标签、`doc_type` 和审核状态等元数据，并写入 `maintenance_knowledge.json`。文档 ID 由源文件名、序号和标题摘要生成，使检索结果能够回溯到具体资料来源，同时为向量索引和图谱构建提供一致的数据标识。

### 3.5.3 同步与索引构建流程

系统使用文件 MD5 记录手册同步状态，并在服务启动时比较当前文件与上次记录；检测到变化后，通过后台线程触发知识同步。当前 `sync()` 会重新解析正式手册，并将审核案例与人工修正一并写入知识 JSON，随后重建 JSON 图谱、DashVector 与 FAISS 索引，并刷新对话检索链。

因此，MD5 主要用于判断是否需要启动同步，而非在同步过程中仅重算单个文件。同步完成后会更新文件哈希、文档数量和错误信息，监控模块可据此展示最近同步状态；向量服务未配置或异常时，本地索引与关键词检索仍可承担基础召回。

#### 3.5.4 申请审核流程

普通用户可通过 `/knowledge/manuals/request` 提交手册申请，文件进入待审核目录并保留申请状态。具备审核权限的管理人员可查看申请列表，根据业务需要执行通过或拒绝操作，并记录审核结果。

审核通过后，文件进入正式手册目录并参与后续知识同步；审核拒绝则保留申请记录但不进入正式知识库。该流程将一线资料补充与正式知识发布分离，避免未经确认的文件直接影响诊断检索结果。

### 3.6 用户与权限管理模块设计

#### 3.6.1 模块功能概述

用户与权限模块采用“权限组基础权限 + 用户额外权限”的控制方式。当前默认权限组包括基础访客、维修人员和管理人员，并支持管理员新增、编辑或删除权限组；用户账号持久化于 MySQL，权限组配置保存在 `permission_groups.json` 中。

#### 3.6.2 权限体系设计

基础访客主要具备对话和图谱查看能力，维修人员增加维修报告

提报能力，管理人员覆盖手册上传、图谱更新、审核等管理权限。管理员还可按实际岗位为单个用户增加额外权限，实现权限组统一配置与个体授权并存。

|        |                                 |                  |
|--------|---------------------------------|------------------|
| 维修人员   | chat、submit_report              | 诊断、SOP与维修报告提报    |
| 管理人员   | chat、view_graph、submit_report 等 | 用户、知识、图谱、审核与系统管理 |
| 自定义权限组 | 管理员配置 permissions               | 按企业岗位创建/调整权限组    |
| 用户额外权限 | extra_permissions               | 在所属权限组基础上补充个体授权  |

图 2 系统角色与权限配置

`UserService` 登录时读取用户所属权限组，并将基础权限与用户 `extra_permissions` 合并为最终权限列表。权限组成员关系由用户当前 `group_name` 动态计算，不需要维护独立的固定组织树，因此可以在不改动业务模块的情况下调整岗位授权范围。

前端依据权限列表控制相关菜单和操作入口，后端管理接口继续执行服务端鉴权。管理员接口优先验证 `JWT` 中的管理员标识，并兼容 `X-Admin-Token`；涉及用户、权限组、知识管理及模型配置的关键操作均由后端再次校验。

### 3.6.3 账号管理流程

系统初始化时创建演示所需的默认账号，当前默认用户分别归入管理人员、维修人员和基础访客权限组。管理员可新增用户、修改所

属权限组、配置额外权限或删除非管理员账号，账号与在线状态统一保存在 MySQL 中。

登录接口校验账号后签发 JWT，并同时写入 HttpOnly Cookie；普通用户 Token 默认有效期为 24 小时，管理员为 12 小时。前端保存用户身份和权限用于界面控制，后端则依据 JWT 对受保护接口进行实际授权；登出时同步清理登录状态。

## 3.7 移动端支持模块设计

### 3.7.1 模块功能概述

为适配现场检修环境的复杂性与一线工人移动办公的需求，系统同步开发了手机 APP，作为桌面 Web 端的有力补充。手机 APP 端与后端服务共用同一套 API，实现核心功能的轻量化迁移，让检修人员在不便使用电脑的产线旁、设备间或户外场景下，仍能随时获取智能检修支持。

### 3.7.2 功能范围与接口复用

手机 APP 端聚焦以下核心能力：支持自然语言提问与故障图片拍摄上传，实现与 Web 端一致的多模态智能问答；支持会话列表查看与历史对话回溯，方便现场快速查阅此前排查记录；支持设备型号筛选，帮助工人精准定位特定机型的知识内容。手机 APP 后端部分复用主系统的 RAG 检索与模型推理服务，确保答案质量与 Web 端保持一致，不因终端变化而折损智能水平。

采用手机 APP 原生框架开发，适配移动端触屏交互特性，优化输入框与图片上传的操作便利性，实现相册选择与拍照即时上传。

### 3.7.3 数据同步与权限适配

手机 APP 权限体系与 Web 端完全一致，通过登录接口返回的权限列表控制小程序端各功能入口的显示与可用性。

手机 APP 特别适用于一线工人“边走边问”“即拍即查”的典型场景，例如在巡检途中发现异常立即拍照查询，或在工作间隙快速回顾以往故障处理方案。系统通过手机 APP 将智能检修能力延伸至车间最前沿，有效缩短了从发现问题到获取解决方案的时间窗口，进一步提升整体排障效率。

## 3.8 国产化适配与部署模块设计

### 3.8.1 模块功能概述

国产化适配模块面向 LoongArch64 与银河麒麟等国产 Linux 环境，重点解决前端运行时、Python 依赖、数据库及系统组件在目标架构上的安装与启动问题。项目已按照目标环境完成适配部署，并将国产化运行作为当前版本的工程基础，而非后续规划。

### 3.8.2 架构兼容性设计

前端采用 Vue 3 + Vite, 可使用 LoongArch64 原生 Node.js 运行; 后端采用 Python 3.11+ 与 FastAPI, 可通过系统虚拟环境安装依赖; 数据库使用 MySQL 或 MariaDB 原生版本。部署方案优先采用目标系统包和本机编译依赖, 不以固定 x86 Docker 镜像作为默认启动方式。

### 3.8.3 信创环境测试与验证

在目标环境联调中, 前后端服务、数据库连接、智能问答、知识图谱、SOP、维修记录与知识同步等核心流程均按照同一套接口运行。模型 API、MySQL、DashVector 等外部服务通过 .env 配置, 迁移到国产服务器时仅需按目标网络与服务地址重新配置。

系统现有监控能力可查看数据库连接、知识目录、向量索引、模型配置及最近同步状态, 用于适配环境下的运行检查。对于包含本地编译扩展的 Python 依赖, 需要使用 LoongArch64 对应编译工具链和头文件完成安装, 以保持运行环境与处理器架构一致。



## 第四章 接口设计

### 4.1 前后端 API 接口设计

系统前后端通过 HTTP 接口通信，普通响应采用 JSON，多轮诊断采用 SSE，图片上传采用 multipart/form-data。用户登录后使用 JWT 访问受保护功能；管理员接口优先校验 JWT，同时兼容 X-Admin-Token。接口按对话、知识、维修、用户和监控等业务域划分，便于前后端独立联调。

#### 4.1.1 智能对话类接口

POST /chat/stream 为 RAG 流式诊断接口，接收 user\_id、session\_id、question、device\_model 和 image\_url 等参数并返回 SSE。响应除文本外还可包含知识来源和 SOP 版本信息，用于前端同步展示诊断依据与作业步骤。

POST /chat/stream/multipart 支持直接上传故障图片；当前实现检测到图片后转入 Agent 诊断路径。POST /chat/agent 则用于复杂故障的多工具诊断，可在 SSE 中返回 tool\_start、tool\_end、sop\_state 等事件，使前端展示 Agent 的检索与状态更新过程。

#### 4.1.1 智能对话类接口

| 接口                          | 请求方式           | 核心响应 / 用途                                       |
|-----------------------------|----------------|-------------------------------------------------|
| POST /chat/stream           | JSON           | RAG 流式诊断；返回 text / sources / sop_version / done |
| POST /chat/stream/multipart | FormData + SSE | 上传故障图片；有图片时进入 Agent 诊断路径                        |
| POST /chat/agent            | JSON + SSE     | LangGraph 多工具诊断；返回工具调用与 SOP 状态                  |

图 3 智能对话类接口

#### 4.1.2 知识库类接口

GET /knowledge/graph 获取结构化知识图谱，可通过 entity\_type 筛选设备、部件、故障、原因和解决方案；GET /knowledge/graph/node/{biz\_id} 返回单个节点详情，GET /knowledge/graph/stats 返回图谱统计信息。

POST/knowledge/graph/extrac、  
POST/knowledge/graph/extract/{doc\_id} 用于管理员触发全量或指定文档的三元组抽取。

POST /knowledge/sync 用于同步正式手册、审核案例和人工修正，并重建知识 JSON、图谱及向量索引。同步结束后刷新 RAGChain，使新增知识无需重启后端即可参与后续检索。

GET /knowledge/manuals 用于获取手册文件、类型、大小、同步状态和文档数等信息；POST /knowledge/manuals/upload 供管理员直接上传正式手册，并可结合同步流程更新知识库。

DELETE /knowledge/manuals/{filename} 用于删除指定手册；系统还提供手册内容、预览及 PDF 状态诊断相关接口，用于管理端检查

文件是否可读取、是否存在文字层及当前解析状态。

POST /knowledge/manuals/request 用于普通用户提交手册申请，文件先进入待审核区域，不直接参与正式检索。

GET /knowledge/manuals/requests 与对应审核接口用于查看、筛选和处理手册申请，审核通过后文件进入正式资料目录。

POST/knowledge/cases 、 案 例 审 核 接 口 及 POST /knowledge/corrections 用于提交维修案例和人工修正，使现场经验在审核后能够进入动态知识库。

4.1.2 知识库与图谱类接口

| 接口                                 | 权限 / 参数        | 说明                |
|------------------------------------|----------------|-------------------|
| GET /knowledge/graph               | entity_type 可选 | 图谱节点、关系、分类        |
| GET /knowledge/graph/node/{biz_id} | 路径参数           | 节点详情与邻接关系         |
| POST /knowledge/graph/extract      | 管理员            | LLM 三元组抽取并写入结构化图谱 |
| POST /knowledge/sync               | 管理员            | 同步手册/案例/修正并重建索引   |
| GET /knowledge/manuals             | 管理员            | 手册列表、大小、同步与解析状态   |
| POST /knowledge/manuals/upload     | 文件上传           | 管理员上传正式手册         |
| POST /knowledge/manuals/request    | FormData       | 普通用户提交待审核手册       |
| POST /knowledge/cases              | FormData       | 提交维修案例，进入审核流程     |

另提供申请审核、人工修正、图谱统计及 Neo4j AuraDB 扩展查询接口。

图 4 知识库与检索类接口

#### 4.1.3 用户管理类接口

**POST /user/login** 校验账号后返回 JWT、用户名、权限组和权限列表，并写入登录 Cookie；登录接口本身无需管理员权限。

**POST /user/logout** 用于注销当前用户并更新在线状态，同时清除登录 Cookie。

**GET /user/list** 返回用户、权限组、在线状态及权限信息，仅管理员可访问。

**POST /user/add** 新增用户，**PUT /user/{username}/group** 调整用户所属权限组，均需管理员权限。

**PUT /user/{username}/permissions** 用于配置用户额外权限；  
**/user/groups** 相关 **GET**、**POST**、**PUT**、**DELETE** 接口用于权限组的查询与维护。

**DELETE /user/{username}** 用于删除非管理员用户，系统对默认管理员账号设置删除保护。

4.1.3 用户与权限管理接口

| 接口                               | 鉴权 / 请求 | 说明              |
|----------------------------------|---------|-----------------|
| POST /user/login                 | 用户名、密码  | 返回 JWT、权限组、权限列表 |
| POST /user/logout                | 用户名     | 注销并清理登录 Cookie  |
| GET /user/list                   | 管理员     | 用户、在线状态与权限      |
| POST /user/add                   | 管理员     | 新增用户并指定权限组      |
| PUT /user/{username}/group       | 管理员     | 调整用户所属权限组       |
| PUT /user/{username}/permissions | 管理员     | 配置用户额外权限        |
| GET/POST/PUT/DELETE /user/groups | 管理员     | 权限组查询与 CRUD     |
| DELETE /user/{username}          | 管理员     | 删除非管理员用户        |

图 5 用户管理类接口

4.1.4 系统监控类接口

GET /monitor/status 返回系统版本、运行时长、知识库文档统计、数据库连接、知识目录、向量索引及同步状态等信息，仅管理员可访问。

POST /monitor/test-llm 用于实际测试大模型连通性；GET /monitor/model-config 与 PUT /monitor/model-config 用于读取或更新聊天、视觉、Embedding、ASR 与 TTS 等模型配置。

4.1.4 系统监控与模型配置接口

| 接口                        | 鉴权      | 用途                                          |
|---------------------------|---------|---------------------------------------------|
| GET /monitor/status       | 管理员 JWT | 系统、知识库、数据库、索引与同步状态                          |
| POST /monitor/test-llm    | 管理员 JWT | 实际测试大语言模型连通性                                |
| GET /monitor/model-config | 管理员 JWT | 读取 Chat / Vision / Embedding / ASR / TTS 配置 |
| PUT /monitor/model-config | 管理员 JWT | 更新指定模型、Base URL 与 API Key                   |

图 6 系统监控类接口

## 4.2 模块间内部接口设计

系统内部通过服务方法与 REST 接口共同完成模块协作。前端主要调用公开 API，后端内部则由 RAGChain、Agent、SopService、KnowledgeSyncService、维修记录服务和用户服务相互协同，减少跨层直接访问数据存储。

`/user/login`: POST + JSON，完成账号校验并签发 JWT，返回用户和权限信息。

`/chat/stream`: POST + SSE，执行 RAG 流式诊断并返回文本、来源及 SOP 信息。

`/chat/agent`、`/chat/stream/multipart`: POST + SSE/FormData，承担 Agent 多工具诊断与图片输入。

`/knowledge/graph`: GET + JSON，获取图谱并支持 `entity_type` 筛选；图谱查询在 MySQL 异常时可回退 JSON。

`/knowledge/sync`: POST + JSON，统一同步手册、动态知识、图谱和向量索引。

`/knowledge/manuals`、`/knowledge/manuals/request`: 分别承担正式手册管理与普通用户资料申请。

`/maintenance/records`: 提供维修记录分页查询、新增、修改、删除与 CSV 导出等能力。

`/maintenance/reports/sync`、`/maintenance/records/submit-report`: 用于维修报告持久化与提报状态管理。

`/user/list`、`/user/groups`: 用于管理员维护用户、权限组和成员授权。

/monitor/status、/monitor/model-config：用于系统状态检查与模型服务配置管理。

4.2 模块间主要接口与服务关系

| 接口                        | 协议              | 职责             | 主要服务                       |
|---------------------------|-----------------|----------------|----------------------------|
| /chat/stream              | POST + SSE      | RAG诊断          | 前端工作台                      |
| /chat/agent               | POST + SSE      | Agent多工具诊断     | LangGraph Agent            |
| /knowledge/sync           | POST + JSON     | 知识同步与索引重建      | KnowledgeSyncService       |
| /knowledge/graph          | GET + JSON      | 图谱查询           | GraphDBService / JSON回退    |
| /maintenance/records      | REST + JSON     | 维修记录 CRUD / 导出 | MaintenanceRecord          |
| /maintenance/reports/sync | GET/POST + JSON | 维修报告持久化        | MySQL                      |
| /user/list、/user/groups   | REST + JSON     | 用户与权限组管理       | UserService / GroupService |
| /monitor/status           | GET + JSON      | 运行状态监控         | Monitor API                |

图 7 模块间内部接口

## 第五章 非功能性设计

### 5.1 性能设计

#### 5.1.1 异步任务处理机制

系统将耗时工作尽量移出主请求链路。服务启动时先比较手册文件 MD5 与上次同步状态，仅在检测到变化后启动后台线程执行知识同步，避免阻塞 FastAPI 启动；维修记录新增或更新后，也可通过 BackgroundTasks 触发案例同步。实际 sync() 过程会重新解析知识并重建索引，以保证知识文件、图谱和向量数据保持一致。

#### 5.1.2 检索性能优化

检索链采用案例优先策略：命中维修案例时直接复用案例并检索手册补充；否则执行向量、关键词和报警代码检索，再按统一评分合并进行图谱扩展。DashVector 提供主要语义检索能力，本地 FAISS 作为备用索引，关键词通路在向量服务异常时仍可工作，从而兼顾检索质量与可用性。

#### 5.1.3 前端渲染优化

前端通过 SSE 增量渲染长文本和 Agent 工具事件，避免等待完整响应后一次性更新。知识图谱页面将最大可视节点数控制为 800，超出后按连接度保留高关关节点，并结合 entity\_type 筛选和 ECharts large 模式降低力导向布局的渲染压力。



## 5.2 可靠性设计

### 5.2.1 多级容错降级机制

系统对关键外部能力设置分层降级。**RAG** 流式生成异常时回退至非流式生成；当生成结果仍不可用时，可直接返回知识库中排名靠前的匹配内容。向量召回异常时，关键词检索继续提供结果，知识同步同时维护本地 **FAISS** 索引。图谱查询在 **MySQL** 不可用时还可回退至 **JSON** 图谱，降低单点服务异常对基础功能的影响。

### 5.2.2 数据备份与恢复策略

当前版本的核心数据分别持久化于 **MySQL** 和知识目录，具备开展备份与恢复的明确边界。生产部署时应对 **MySQL** 执行周期性逻辑或快照备份，并同步备份 **data/**、**knowledge/** 及环境配置文件；恢复时先还原数据库与知识文件，再重新执行知识同步以重建图谱和向量索引。文档不将尚未实现的自动备份调度描述为现成功能。

### 5.2.3 日志与监控

后端使用日志记录数据库初始化、知识同步、对话、**Agent** 与异常信息；管理端通过 **/monitor/status** 汇总数据库、知识目录、向量索引、模型配置和最近同步状态，并提供大模型连通性测试。日志与监控结合，可帮助运维人员区分数据库、知识索引、模型服务或文件路径问题。

## 5.3 可扩展性设计

### 5.3.1 插件化模型接入

模型服务采用统一配置与 **Provider** 封装。聊天模型、视觉模型、**Embedding**、**ASR** 和 **TTS** 均可通过环境变量或监控接口维护，其中聊天模型使用 **OpenAI** 兼容调用方式。业务层主要依赖统一的模型获取接口，因此切换兼容模型时无需修改具体诊断流程。

### 5.3.2 可扩展的节点与文档类型

文档解析入口根据文件后缀分发到 **PDF**、**DOCX** 和 **XLS/XLSX** 解析逻辑，新增资料类型时可在 `parse_manual_file` 中扩展对应解析器。图谱接口通过 `entity_type` 区分不同实体，并将 **JSON**、**MySQL** 与 **Neo4j** 查询封装在独立服务中，为后续扩展节点类型或图数据库实现保留清晰边界。

### 5.3.3 部署架构可扩展

系统采用前后端分离结构，模型、数据库和向量服务均通过配置解耦，可根据部署规模独立迁移或替换。前端和后端使用标准 **HTTP** 接口协作，知识数据与业务数据分层存储；因此在扩充服务器或调整外部服务时，核心业务模块无需整体重写。当前版本以明确的服务边界为扩展基础，不额外宣称尚未实现的自动集群编排能力。

## 5.4 安全设计

### 5.4.1 身份认证与授权

用户登录成功后由后端签发 HS256JWT，普通用户默认 24 小时有效、管理员 12 小时有效。JWT 可通过 Authorization 请求头或 HttpOnly Cookie 携带；管理员接口优先校验 JWT 中的管理员标识，并兼容旧版 X-Admin-Token。前端权限列表用于控制交互入口，真正的管理操作仍由后端再次鉴权。

### 5.4.2 数据安全

用户、会话、SOP 和维修记录等业务数据持久化于数据库，知识文件和权限组配置存放在受控目录，模型密钥及数据库连接信息通过 .env 管理。若采用云端大模型或向量服务，检索上下文和向量请求会离开本机，因此实际部署应依据企业数据分级选择云端或私有化服务，并通过网络白名单、最小权限账号和配置隔离控制数据暴露范围。

## 5.5 国产化环境适配设计

### 5.5.1 架构兼容性约束

系统主要采用 Vue/TypeScript 与 Python/FastAPI 等跨平台技术，避免在业务代码中直接依赖 x86 特有指令。LoongArch64 环境下应优先安装对应架构的 Node.js、Python、MySQL 与系统库；包含本地编译扩展的 Python 包需使用目标发行版的编译工具链构建。

### 5.5.2 操作系统内核依赖

系统不依赖非标准 Linux 内核模块,路径和服务启动方式遵循常规 Linux 环境。国产化部署优先使用系统 Python 虚拟环境和原生 Node.js 包;若 uv 在目标环境不可用,可使用 Python 3.13 venv 与 pip 安装项目依赖。当前方案不把固定 x86 Docker 镜像作为默认启动前提。

### 5.5.3 网络与通信约束

外部模型、DashVector、Neo4j AuraDB 等云服务需要目标环境具备相应网络访问能力,连接地址和密钥均由 .env 或管理配置维护。企业可按网络安全要求限制出站域名,并在需要降低外网依赖时替换为本地模型、MySQL/JSON 图谱和 FAISS 等本地能力;所有服务端口与访问策略应纳入目标环境防火墙管理。